Security Audit Report

BTC Grow

Date September 30, 2025
Project BTC Grow
Version 0.0.0

Contents

Disclosure 1
Disclaimer 2
Scope 4
Assessment Overview 5
Assessment Components 6
Executive Summary 7
Code Base 10
Severity Classification 11
Finding Severity Ratings 12
Findings 13
[C 501] Router reward farming via micro-withdrawals 15
[H 401] `babel_fees` Funds DDoS/Contention 16
[H 402] Malicious `WithdrawFromFloat` 17
[M 301] Mixed-unit comparison in transfer check 18
[M 302] Incompatible transfer_amount constraints vs Danogo 20
[M 303] Unsafe precondition for division 22
[M 304] Config reference UTxO update contention 23
[L 201] Blind Transaction/Metadata Signing 25
[L 202] Incentivize `SupplyToFloat` Router Batching 26
[L 203] Static wBTC withdrawal fee misprices ADA costs 27
[L 204] Min-transfer invariant only enforced at final phase 29
[I 101] Inefficient self-withdraw validation 30
[I 102] Split withdraw variants into separate validators 32
[I 103] Precompute Formatted Address 33
[I 104] Redundant Redeemer Verification 35
[I 105] Possibly Inaccurate Transfer Amount 37
[I 106] Redundant Boolean Return 38
[I 107] Redundant wBTC check 40
[I 108] Optimize On-chain Transaction CBOR Creation 42
[I 109] Simplify Reference Input List Iteration 43
[I 110] Avoid per-UTxO scan in `bridge_asset.spend` 45

Disclosure

Nonetheless, both the customer **Danogo** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this audit was limited to the code provided. It did not extend to compiler-level artifacts (e.g., generated UPLC code) or to areas beyond the explicitly reviewed codebase. Additional testing methodologies, including unit testing and property-based testing, were also outside the scope of this engagement.

External bridging dependency (Wanchain)

Bridging BTC → Cardano via Wanchain (WBTC arriving at `bridge_asset`) introduces risks that are outside the scope of BTC Grow's Cardano validators. Assuming correct off-chain submission by BTC Grow and absent a Wanchain-side incident, notable failure modes include:

- WBTC not sent to the `bridge_asset` validator address.
- WBTC sent with an incorrect or missing staking credential.
- WBTC sent with an unexpected datum
  - (e.g., `BridgeAssetSupplyDatum { message: constants.danogo_magic_message }`).
- **Note:** the last case can be mitigated with additional on-chain handling in
  - `lib/bridge_asset/withdraw_float.ak`.

Assurance/refund mechanisms, if any, are external to this audit and out of scope.

Protocol flow dependency (Float availability)

Normal flow:
`wanchain-bridge → bridge_asset → Float → bridge_asset → wanchain-bridge`

If the Float protocol is unavailable or supply cannot proceed, a direct return path `wanchain-bridge → bridge_asset → wanchain-bridge` does not exist. In such cases, WBTC cannot be returned to the owner via BTC Grow alone; funds remain held until the Float interaction can be completed. This is a known/accepted protocol limitation (as per client correspondence) and is documented here rather than as a finding.

Operational centralization (current phase)

The `ProtocolConfigDatum` is currently operator-controlled. This means that users must rely on the protocol operators to manage parameters responsibly. In particular, if the protocol stalls under the conditions noted above, operators can update `float_pool_nft` (and related configuration fields) to produce favorable datum values and unwind/return user funds. The `ProtocolConfigDatum` plays a crucial role in the operational management of the protocol, and its control by the operators is a key aspect of the protocol's current phase.

This serves as an operational mitigation for certain edge cases; however, it is not a trustless guarantee and depends on operator keys, procedures, and timeliness. (Also note that config updates may briefly contend with user txs; see findings regarding reference-UTxO updates.)

Dependency on Float protocol state

BTC Grow's behavior depends on the external state and parameters provided by the Float protocol. The correctness, liveness, and economic outcomes of BTC Grow transactions therefore rely on the availability, timeliness, and consistency of that external state, as well as on the compatibility of interpretation between systems. Variations or updates to Float may alter outcomes, result in temporary rejections, or shift economic effects. These dependencies are acknowledged as design constraints and are documented here as part of the disclaimer rather than as individual findings.

Scope

In scope

- On-chain validators and supporting logic for BTC Grow at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

Out of scope

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

Assessment Overview

From **August 25**, **2025** to **September 25**, **2025**, **Danogo** engaged No Witness Labs to conduct a comprehensive security assessment of the **BTC Grow** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

Assessment Components

Manual revision

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention

Executive Summary

Project Overview

BTC Grow is a decentralized protocol designed to enable Bitcoin holders to earn yield through a Cardano-based non-custodial lending mechanism. Native BTC is locked via Wanchain, represented as wBTC on Cardano, and routed into the Float lending pool to generate interest. Users can withdraw at any time, with redemptions handled through a two-phase process: withdrawal from Float, and transfer back via Wanchain.

The system relies on several core validators:

- **Bridge Asset** – manages wBTC representation, supply, withdrawal, and redemption.
- **Babel Fee** – provides ADA to cover router transaction costs.
- **Metadata Validation** – enforces the integrity of transaction metadata.

Operationally, BTC users interact by locking or withdrawing BTC, while router agents assemble and submit Cardano transactions on their behalf, compensated through Babel fee UTxOs. The License NFT grants administrative authority over Babel fee withdrawals.

The audit focused exclusively on the Cardano on-chain logic. Dependencies on external systems, including Wanchain bridging and the Float lending pool, were acknowledged but remained outside the scope of this review. As with all DeFi systems, residual risks exist, and security assumptions rely on both audited Cardano contracts and the correct operation of external protocols.

Extended Overview

Design Overview

BTC Grow introduces a non-custodial staking flow for Bitcoin that integrates Wanchain bridging and Cardano-based lending. The design ensures that native BTC is locked via Wanchain's decentralized bridge, while its Cardano counterpart (wBTC) is used to generate yield.

Key characteristics of the design include:

- Liquid staking: no lock-ups; BTC can be withdrawn at any time.
- On-chain governance: critical parameters are maintained in Protocol Config UTxOs, identified by NFTs.
- Fee handling: Transparent fee handling is achieved through the Babel fee system, which covers transaction costs, router rewards, and Wanchain bridge fees. In addition, users pay a protocol fee defined in the Protocol Config, which is operator-controlled and subject to change.

Roles and Responsibilities

- BTC User: Provides the BTC and controls withdrawal through ECDSA signatures from their Bitcoin wallet.
- Router Agent: Submits transactions on Cardano. Agents are permissionless and compensated in ADA via Babel fee UTxOs.
- Admin / License NFT Holder: Authorized to withdraw accumulated ADA from the Babel fee UTxO.

Lifecycle of Assets

1- Lock & Supply:

- BTC is locked on Wanchain and minted as wBTC.
- Router agents submit a transaction that moves wBTC into the Float lending pool, referencing dToken exchange rates.
- Multiple UTxOs can be supplied in a single transaction.

2- Withdraw from Float:

- Users initiate a withdrawal request with an ECDSA signature.
- Router agents redeem dwBTC into wBTC, producing up to two outputs:
  - A Remain UTxO (holding any residual dwBTC).
  - A Transfer UTxO (wBTC prepared for burning).

3- Redeem & Transfer to Wanchain:

- Router agents burn the wBTC in the Transfer UTxO.
- Wanchain monitors burns and releases the equivalent native BTC back to the user's wallet.
- Outputs in this phase include protocol fees and Wanchain fees.

Supporting Infrastructure

- Bridge Asset Validator: Governs wBTC UTxOs, enforcing constraints for supply, withdrawal, and redemption flows.
- Babel Fee Validator: Handles optional fee UTxOs that allow routers to be compensated without bearing transaction costs.
- Metadata Validation Script: Confirms transaction metadata consistency by reconstructing and hashing transaction bodies, protecting against malformed or manipulated off-chain metadata.
- Parameterized Deployment: Uses NFTs (protocol script NFT, config NFT, license NFT) to anchor script identity, configuration authority, and administrative control.

Risks and Dependencies

- Bridge dependency: Security assumptions depend on Wanchain's decentralized bridge network.
- Float dependency: Yield generation requires Float lending pools; protocol liveness depends on Float availability.
- Operational centralization: Protocol Config UTxOs are currently operator-controlled; this may evolve as governance matures.
- Fee variability: Protocol and Wanchain fees must be updated as market conditions (e.g., BTC/ADA exchange rates) change.

Code Base

Repository

https://github.com/Danogo2023/btc-grow.git

Commit

2dc5288af8ba938f339b2d543e0da09987307aa8

Files Audited

Files	SHA256 Checksum
lib/babel_fee/supply_float.ak	a38bf9639caf953471a8964b636f389d24ab5a18b15bbb127e533fc696c71ad0
lib/babel_fee/transfer_wanchain.ak	09e692734223b342329c82666f4850e6b87bae6d0b6e54a20f631a427fb1cb39
lib/babel_fee/utils.ak	d012e982f33afe09a5b47d0e18a0f5e66fc487b62f3a6f44db0f9fd81d73fb86
lib/babel_fee/withdraw_float.ak	687445c7e8f387eaede311ebc25db0fa4f1c913f5d7eb1595e37f31dacfcd06f
lib/bridge_asset/supply_float.ak	d9e605bbdae69a65e35f6d14474b742239a9a2797ce8355c69e456f68a457283
lib/bridge_asset/transfer_wanchain.ak	14aca2919e43cd34d62d4fb7c68ea42b7c8453a1f44fdf89b6726eae1388b3e5
lib/bridge_asset/withdraw_float.ak	ee96ff64b982e5bd198e8dee60f198ce708c7ff608da2bc0bc7322dc2d21fd56
lib/metadata_validation/utils.ak	97973fb5bb4347e82b16119f1eb6de2d19d34bbe752f824db4911b7f7f28515e
lib/metadata_validation/validate.ak	117954c06786cd0f3f249bafb0dea70b8bb9567a211019f00b502c1cde8b4bc1
lib/constants.ak	62e82541878c0e9751006b542574ce209a3e3306fe5fdce926e8f481db7e92c0
lib/types.ak	bb4ef771a295c67c862b377acba7fd5623ee81934008ddaf777d12c51b9e978b
lib/utils.ak	7f166a9b72e89d400a2bae7a952e74bc42a2d8e815dca0a9c8f8f6e25f0bc361
validators/babel_fee.ak	32321ea1f063293f2a36910993cf0e2c0cca89e4923fe26a62fc7e9d6128a0a9
validators/bridge_asset.ak	e2af981b3187d93dc682d92bf31ad19905a0e79585bc20aae7f5027bc9289d64
validators/metadata_validation.ak	3b5fb7039df6f60f6ebcf272b712500d1cc1497783b05eb43a9b7cf5f6a35e92

Severity Classification

- **Critical**: Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High**: Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium**: Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low**: Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational**: Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

Level	Severity	Status
5	Critical	1
4	High	2
3	Medium	4
2	Low	4
1	Informational	10

Findings

ID	Title	Severity	Status
501	Router reward farming via micro-withdrawals	Critical	Resolved
401	`babel_fees` Funds DDoS/Contention	High	Partially Resolved
402	Malicious `WithdrawFromFloat`	High	Resolved
301	Mixed-unit comparison in transfer check	Medium	Resolved
302	Incompatible transfer_amount constraints vs Danogo	Medium	Acknowledged
303	Unsafe precondition for division	Medium	Resolved
304	Config reference UTxO update contention	Medium	Acknowledged
201	Blind Transaction/Metadata Signing	Low	Acknowledged
202	Incentivize `SupplyToFloat` Router Batching	Low	Acknowledged
203	Static wBTC withdrawal fee misprices ADA costs	Low	Acknowledged
204	Min-transfer invariant only enforced at final phase	Low	Resolved
101	Inefficient self-withdraw validation	Informational	Enhanced
102	Split withdraw variants into separate validators	Informational	Acknowledged
103	Precompute Formatted Address	Informational	Resolved

ID	Title	Severity	Status
104	Redundant Redeemer Verification	Informational	Resolved
105	Possibly Inaccurate Transfer Amount	Informational	Acknowledged
106	Redundant Boolean Return	Informational	Resolved
107	Redundant wBTC check	Informational	Resolved
108	Optimize On-chain Transaction CBOR Creation	Informational	Acknowledged
109	Simplify Reference Input List Iteration	Informational	Resolved
110	Avoid per-UTxO scan in `bridge_asset.spend`	Informational	Deprioritized

[C 501] Router reward farming via micro-withdrawals

Level	Severity	Status
5	Critical	Resolved

Description

> `lib/bridge_asset/withdraw_float.ak`

The router is paid a fixed 2 ADA reward per withdrawal, while protocol fees are only charged later at the transfer stage. Because the withdrawal amount can be as low as 1 dWBTC (≪ 1 ADA), an attacker acting as the router can repeatedly perform micro-withdrawals to convert trivial dWBTC into 2 ADA per call, profiting after transaction fees and draining the ADA reward pot.

External Dependency Note: *Danogo currently enforces a minimum withdrawal size. However, relying on external protocol invariants is outside our trust boundary. This validator must remain secure even if that minimum is reduced, reconfigured, or bypassed via composition.*

Key conditions enabling the exploit

- Fixed router reward (2 ADA) independent of withdrawn dWBTC amount.
- No minimum dWBTC withdrawal enforced.
- Protocol fees not collected (or not escrowed) at the withdrawal stage.

Attack Scenario

1. Attacker becomes (or front-runs as) the router.
2. Calls withdraw_float with amount = 1 dWBTC repeatedly.
3. Receives 2 ADA each time; pays normal tx fee (< 2 ADA), nets profit.
4. Repeats to drain ADA earmarked for router rewards/contract operations.

Impact

- Direct ADA loss from the protocol (Babel fee faucet).
- Liveness degradation when reward funds deplete.
- Economic unfairness for honest users whose fees later fund a depleted pot.

Recommendation (choose one)

- Pay rewards from collected fees at the same step: escrow/charge the user's protocol fee at withdrawal.
- Enforce a minimum withdrawal threshold (in dwBTC) that makes the reward non-profitable for dust — do not rely on Danogo's minimum.
- Alternatively, defer paying the router until TransferToWanchain transaction.

Resolution

Code Fix (On-chain) — added minimum transfer gate `(transfer_amount >= fee_withdraw)` to block micro-withdrawals that farm fixed router rewards; effective when `fee_withdraw` reflects total ADA outlay (router + Wanchain). See F203.
Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[H 401] `babel_fees` Funds DDoS/Contention

Level	Severity	Status
4	High	Partially Resolved

Description

The `babel_fees` validator stores UTxOs with lovelaces for BTC Grow routers to cover their lovelace requirements for protocol specific transactions. Since routers are not permissioned, any actor can become a router and spend funds from `babel_fees` contract. As of now, a single router can spend all the UTxOs at `babel_fees` in a single transaction, even though it's fee requirements are very minimal (2 ADA) as configured in `ProtocolConfigDatum`'s `fee_router`. With no minimum threshold for supply (or withdraw from) to float amount enforced in BTC Grow protocol, a malicious actor satisfying minimum transaction size limit enforced by Float protocol, can repeatedly supply dust wBTC amounts to Float and spend `babel_fees` UTxO denying it's service to other routers, degrading protocol functionality.

The protocol requires the remaining funds to be returned to `babel_fees` validator in a single UTxO. This de-fragmentation of UTxOs will result in less number of UTxOs available for different routers, increasing UTxO contention among them.

Impact

1. Denying access to `babel_fees` funds, affecting routing activity and degrading protocol functionality.
2. De-fragmentation of UTxOs at `babel_fees`, increasing contention amongst routers.

Recommendation

1. The `babel_fees` spending validator should not allow additional UTxOs to be spent, once lovelace requirement of `fee_router + min_ada` is satisfied while iterating over inputs.
2. A minimum transaction amount must be configured for BTC Grow supply and withdraw actions.

Resolution

Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[H 402] Malicious `WithdrawFromFloat`

Level	Severity	Status
4	High	Resolved

Description

`bridge_asset`'s `WithdrawFromFloat` withdrawal validation allows spending of `bridge_asset` UTxOs without checking their datums (allows `BridgeAssetTransferDatum`) or values (can contain WBTC). A compromised/malicious/erroneous off-chain library can take advantage of unrouted Supply or TransferOut UTxOs from a particular user who wishes to withdraw while already having pending unprocessed orders.

Attack Scenario

1. User has an unprocessed `BridgeAssetTransferOutDatum` UTxO (or `BridgeAssetSupplyDatum`/`NoDatum` UTxO) containing WBTC. UTxO's staking credential is user's `btc_pub_key_hash`.
2. User initiates a new transfer out request, perhaps to amend his previous transfer out or supply action.
3. A compromised/malicious/erroneous off-chain library would include the above unprocessed UTxOs output references, not containing any dWBTC, along with other user UTxOs with dWBTC and get user's signature on them.
4. `WithdrawFromFloat` withdrawal validation allows spending of all user output references which the user has signed, irrespective of a UTxO's datum.
5. It calculates the total amount of dWBTC present across all UTxOs and ignores WBTC amount.
6. Expected WBTC amount gets withdrawn from Float protocol after burning dWBTC tokens while already present WBTC tokens are stolen by the malicious router.

Recommendation

Only UTxOs with the expected datum should be spendable via `WithdrawFromFloat`

- `expected_datum == BridgeAssetSupplyDatum { message: constants.danogo_magic_message }`

Also, it is advisable for UTxOs with `BridgeAssetTransferOutDatum` to not have any staking credential.

Resolution

Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[M 301] Mixed-unit comparison in transfer check

Level	Severity	Status
3	Medium	Resolved

Description

> `lib/bridge_asset/withdraw_float.ak` line 156

The transfer validation mixes different units in a single inequality:

```
and {                                                                    Aiken
  (transfer_payment_credential == Script(bridge_asset_skh))?,
  // transfer_amount in wBTC, total_amount in dWBTC
  (transfer_amount >= total_amount)?,
  (transfer_output_value
    |> assets.without_lovelace
    |> assets.add(env.wbtc_pid, env.wbtc_asset_name, -transfer_amount)
    |> assets.is_zero)?,
  (transfer_amount >= expected_transfer_amount)?,
  (transfer_output_datum == InlineDatum(
    BridgeAssetTransferDatum { message: constants.danogo_magic_message,
    btc_receiver_address },
  ))?,
  (transfer_output_reference_script == None)?,
}
```

- **transfer_amount** is denominated in wBTC (the bridged asset).
- **total_amount** is denominated in dwBTC (protocol's derivative token).

Unless the protocol guarantees a strict 1:1 peg (1 dwBTC ≡ 1 wBTC, always), `transfer_amount >= total_amount` is dimensionally incorrect. Its effect depends on the current conversion rate `r = dwBTC_per_wBTC`:

- If r < 1 (1 wBTC is worth fewer dwBTC), this check is too weak but redundant because the code also checks `transfer_amount >= expected_transfer_amount` (which presumably uses the correct conversion).
- If r > 1, this check is too strict: even when `transfer_amount == expected_transfer_amount`, the mixed-unit guard may still fail and reject valid transactions (a DoS on users).

The presence of the correct guard `transfer_amount >= expected_transfer_amount` makes the mixed-unit check both brittle and unnecessary, and it can break once rates deviate from 1:1.

Recommendation

Remove the mixed-unit guard:

- Delete `(transfer_amount >= total_amount)?`. Rely on the rate-aware invariant already expressed by `expected_transfer_amount`.
- Or normalize both sides to the same unit (if you want a single explicit check).

- Or check `total_amount` against the -minted (burned) dwBTC: `total_amount == -mint_dwBTC`

(Optional) Type-Safe Units

- Use unit-specific newtypes (e.g., WBTC, DWBTC, rate type) for monetary calculations.
- Avoid raw Int to prevent mixed-unit errors.
- Performance: negligible overhead when unwrapped once at boundaries.

Document unit invariants:

- Comment the units for `transfer_amount`, `total_amount`, and how `expected_transfer_amount` is computed (including rounding), and add property tests covering both r < 1, r == 1, and r > 1.

Impact

- Prevents unintended DoS when the conversion rate deviates from 1:1.
- Removes a brittle, redundant check (safer and easier to maintain).
- Makes unit semantics explicit, reducing future integration bugs.

Resolution

Code Fix (On-chain) — removed mixed-unit guard `transfer_amount ≥ total_amount` and enforced a single rate-aware check using `expected_transfer_amount`; all comparisons now occur in consistent units.
Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[M 302] Incompatible transfer_amount constraints vs Danogo

Level	Severity	Status
3	Medium	Acknowledged

Description

> `lib/bridge_asset/withdraw_float.ak`

```
let expected_transfer_amount =
if total_supply != 0 || circulating_dtoken != 0 {
total_amount * total_supply / circulating_dtoken
} else {
expect Some(Input { output: Output { datum, .. }, .. }) =
list.find(inputs, fn(Input { output: Output { value, .. }, .. }) {
utils.contains_nft(value, float_pool_nft)
})
expect InlineDatum(dt) = datum
expect FloatPoolDatum { total_supply, .. } = dt
total_supply
}
```

Aiken

```
(transfer_amount >= expected_transfer_amount)?
```

Aiken

BTC_Grow enforces `(transfer_amount >= expected_transfer_amount)` using a conversion derived from pool parameters, while Danogo's protocol applies additional, independent and indirect constraints on `transfer_amount`. There exist states (now or after Danogo updates) where no single `transfer_amount` can satisfy both BTC_Grow's inequality and Danogo's own checks, causing permanent transaction rejection (DoS) for legitimate users.

Why this is unsafe

- Two disjoint sources of truth (BTC_Grow vs Danogo) govern the same economic quantity.
- Any change in Danogo's rounding/fees/minimums/calculations can make BTC_Grow's guard infeasible.
- Moreover: `expected_transfer_amount` uses integer division (floor), increasing the chance that rounding differences with Danogo make `transfer_amount` unsatisfiable across both systems.

(Trust boundary note: depending on Danogo's current policy or future updates is unsafe; their invariants are outside BTC_Grow's control.)

Impact

- Liveness failure / DoS: Valid withdrawals/transfers become impossible under some pool states or future Danogo releases. *(unsatisfiable transfer_amount ⇒ DoS)*
- Governance/upgrade fragility: Danogo parameter or logic changes can silently brick BTC_Grow flows.

Recommendation

- **Single source of truth:** Compute expected_transfer_amount with the exact same formula and rounding Danogo uses (shared library / code port / versioned parametrization), and enforce equality rather than a loose inequality.
- **Compatibility envelope:** Alternatively, remove BTC_Grow's independent constraint and verify invariants that cannot contradict Danogo (e.g., dToken burn/mint correctness, value conservation, NFT/state linkage), letting Danogo determine the precise transfer_amount.
- **Version pinning:** Bind checks to a config/market version hash read from Danogo's datum; fail if versions mismatch to avoid silent divergence.

Resolution

Risk Accepted — Team elected to rely on Danogo's rate as source of truth; inequality retained. Rationale: any Danogo tx can update the exchange rate and they will trust it. No single-source alignment/pinning implemented.

Residual risk: future changes to Danogo Float (formula/rounding/fees) not mirrored in BTC_Grow can make `expected_transfer_amount` diverge, causing DoS or mispricing.

Monitor and revisit if incidents occur.

[M 303] Unsafe precondition for division

Level	Severity	Status
3	Medium	Resolved

Description

> lib/bridge_asset/withdraw_float.ak

```
let expected_transfer_amount =                                    Aiken
  if total_supply != 0 || circulating_dtoken != 0 {
      total_amount * total_supply / circulating_dtoken
  } else {
      // fallback path...
  }
```

The guard uses OR (||) but then divides by `circulating_dtoken`. If `total_supply != 0` and `circulating_dtoken == 0`, the branch is taken and the code divides by zero, causing the validator to fail (DoS). The zero-liquidity fallback becomes dead in precisely the state where it should be used.

Recommendation

Replace `||` with `&&` in the precondition guarding the division *and* handle `circulating_dtoken == 0` safely.

Impact

- Liveness failure / DoS whenever `circulating_dtoken` == 0 but `total_supply` != 0.
- The intended zero-liquidity fallback is bypassed by the current `||` guard.

Resolution

Code Fix (On-chain) — replaced `||` with `&&` in the precondition guarding division by `circulating_dtoken`; removes the divide-by-zero path and restores the intended zero-liquidity fallback.
Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[M 304] Config reference UTxO update contention

Level	Severity	Status
3	Medium	Acknowledged

Description

> Protocol config held in a reference UTxO (ProtocolConfigDatum).

The protocol reads fees/parameters from a ref input. When operators update the config (to track ADA/BTC, fee changes, etc.), they must spend and recreate that UTxO. During this window:

- User txs referencing the old ref will conflict with the update tx (cannot spend and reference the same UTxO in one tx; ordering in the same block is not guaranteed for clients in mempool).
- Some clients may build against the stale config and get rejected post-ordering, creating a liveness problem.
- If frequent updates are needed (e.g., price volatility), the protocol experiences repeated contention and intermittent DoS for users.

Cardano specifics:

- A UTxO cannot be both spent and referenced in the same transaction.
- In the same block, a later tx can reference the new UTxO produced by an earlier tx, but mempool participants cannot rely on this ordering when building transactions.

Impact

- Liveness degradation / DoS: User transactions fail intermittently during updates.
- Operational fragility: More frequent updates → more contention, higher failure rates.
- Stale-config risk: Transactions may validate against outdated parameters if activation isn't gated.

Recommendation

Option 1 – Dual-ref config with atomic activate/swap

Goal: No user-tx contention while editing config.

Structure

- Maintain two reference UTxOs carrying `ProtocolConfigDatum`:
  - Active UTxO: referenced by user txs; only the activation/cleanup flow may spend it.
  - Pending UTxO: never referenced by user txs; can be updated freely.
- Tokenization: Mint a fungible `protocol_config_token` with total supply = 2.
  - Each config UTxO must hold exactly 1 token (enforced on-chain) to guarantee "exactly two" configs exist.
- Tagging: Both UTxOs are identified by the presence of `protocol_config_token`.
- ProtocolConfigDatum extensions:
  - `config_version: Int` – strictly increasing version used for off-chain selection and anti-stale checks.

- `status: Active | Pending` – authoritative role flag; user transactions must only consume/reference the Active one.

User transactions (normal flow)

- Must reference exactly one config UTxO with `status == Active`.
- Validate on-chain: `status == Active` (reject `Pending`).
- Off-chain selection: choose the Active config with the highest `config_version`.

Update flow (no contention)

- Edit Pending: freely update its datum/content; user txs never reference it.
- Activate Pending (Tx A): promote Pending → Active with `config_version = old_active_version + 2`.
- Deactivate old Active (Tx B): spend old Active and recycle as Pending.
- Invariants enforced: always exactly two config UTxOs with statuses `{Active, Pending}`.

Option 2 – Chaining-based activation (operational mitigation)

- Submit Tx A (new config), then immediately build dependent txs referencing the new UTxO (aiming for the same block).
- Limitations: best-effort only; does not remove public-user contention or mempool ordering risk.

This design minimizes contention/DoS during inevitable config updates.

Resolution

Risk Accepted — No code change. Team assesses impact as minimal (retriable liveness blips during config ref updates). Operational guidance: clients/routers auto-retry with backoff and, when possible, chain against the new config UTxO.

[L 201] Blind Transaction/Metadata Signing

Level	Severity	Status
2	Low	Acknowledged

Description

The btc-grow platform requires users to sign transactions for staking and to sign metadata for withdrawal operations without providing clear visibility into transaction details. Users are presented with opaque transaction data that cannot be easily verified, creating a "blind signing" scenario where they authorize transactions without understanding their contents.

During staking operations, users cannot verify that outputs are correctly directed to the staking contract. For withdrawals, users sign metadata containing recipient addresses that could potentially be modified by attackers, redirecting funds to malicious addresses instead of intended recipients.

Impact

- Users cannot verify transaction correctness during staking/withdrawal
- Attacker could modify recipient addresses
- Loss of user funds due to unauthorized transaction modifications

Recommendation

1. Implement Transaction Transparency: Display comprehensive transaction details including all inputs, outputs, addresses, amounts, and fees
2. Add Human-Readable Summaries: Provide clear explanations of what each transaction accomplishes
3. Enable Address Verification: Allow users to confirm recipient addresses before signing

[L 202] Incentivize `SupplyToFloat` Router Batching

Level	Severity	Status
2	Low	Acknowledged

Description

`bridge_asset` validator allows multiple UTxOs with WBTC to be supplied to the Float protocol in a single transaction. Router agents catalyzing this transaction are however rewarded with a fixed `fee_router` lovelace amount (2 ADA) per transaction, irrespective of the number of supply orders (UTxOs) they fulfilled. This misaligns incentives: routers maximize reward minus fees by processing one order per tx, hurting throughput and overall performance.

Recommendation

Pay `fee_router` per supply order fulfilled (each UTxO spent with the `SupplyToFloat` redeemer) within the transaction. Exclude dust WBTC UTxOs from eligibility to prevent abuse and unfair reward extraction.

[L 203] Static wBTC withdrawal fee misprices ADA costs

Level	Severity	Status
2	Low	Acknowledged

Description

> Protocol config is held in a reference UTxO with:

```
pub type ProtocolConfigDatum {
  float_pool_nft: TupleAsset,
  protocol_fee_address: Address,
  wanchain_fee_address: Address,
  fee_router:   Int,   // ADA per call
  fee_wanchain: Int,   // ADA
  fee_withdraw: Int,   // wBTC (protocol fee)
  wanchain_smg_id: ByteArray,
  wanchain_refund_bech32_address: ByteArray,
}
```

Aiken

Docs state `fee_withdraw` (in wBTC) "offsets the ADA fee_wanchain paid by Babel" and "may need to be updated if the ADA/BTC rate fluctuates." Two issues:

- **Volatility risk:** `fee_withdraw` is a static wBTC amount. When ADA/BTC increases, the ADA outlay (Wanchain + router rewards) costs more wBTC than the fixed `fee_withdraw`, creating a systematic subsidy. An attacker can trigger many small withdrawals to drain ADA from the protocol's babel fee treasury.
- **Under-specification:** The text mentions offsetting only Wanchain fees, but router rewards (e.g., 2 ADA per leg, possibly multiple legs) are not included in the stated rationale. If they're not accounted for, the mispricing worsens.
- **Update fragility:** Keeping `fee_withdraw` current via manual updates to the config reference UTxO is operationally brittle (fast price moves). Frequent updates can cause contention and failed transactions.

Impact

- Direct ADA loss (treasury subsidy) when ADA/BTC rises and `fee_withdraw` lags.
- Fairness risk in the other direction: when ADA/BTC falls, users are overcharged.

Recommendation

- Include router rewards: Define the protocol fee to cover Wanchain fees + router fee(s) so withdrawals never subsidize ADA outlays.
- Add a safety margin: Apply a fixed margin on top of the calculated amount to absorb ADA/BTC volatility, so total fees don't run at a loss even if the rate moves between updates.

Resolution

Operational Policy — keep `fee_withdraw` in ProtocolConfig with a margin above Wanchain's conversion fee; adjust during operations as needed. No code change. Note: router rewards not explicitly included; residual risk of ADA subsidy under volatility and update lag remains.

[L 204] Min-transfer invariant only enforced at final phase

Level	Severity	Status
2	Low	Resolved

Description

> lib/bridge_asset/transfer_wanchain.ak

`transfer_wanchain.spend` enforces:

```
1  expect in_value > fee_withdraw
```
Aiken

...but this isn't enforced in earlier phases (SupplyToFloat, WithdrawFromFloat). As a result, UTxOs that pass the first two phases with `in_value ≤ fee_withdraw` will always fail at `transfer_wanchain.spend`, leaving funds stranded (cannot progress).

Impact

- **DoS / stuck funds:** bridge UTxOs that are ≤ fee threshold become unspendable at the transfer step.

Recommendation

Enforce the same invariant at creation time: when producing the transfer-intended UTxO in SupplyToFloat and WithdrawFromFloat, require `in_value > fee_withdraw` (or a helper like `utils.require_min_transferable(in_value, fee_withdraw)`), and keep the check in `transfer_wanchain.spend` for defense-in-depth.

Resolution

Code Fix (On-chain) — enforced a consistent minimum-transferability invariant across all phases (SupplyToFloat, WithdrawFromFloat, and TransferToWanchain).
Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[I 101] Inefficient self-withdraw validation

Level	Severity	Status
1	Informational	Enhanced

Description

> `utils/require_self_withdraw_with_rdmr.ak`, `lib/babel_fee.ak`

The current `require_self_withdraw_with_rdmr(out_ref, tx, rdmr)` traverses all inputs to recover the script hash from the matching input's payment credential before checking the `Withdraw` redeemer. This couples `spend` to input layout and adds unnecessary O(n) cost.

By separating the withdraw logic into its own validator and passing its `ScriptHash` into the main `babel_fee` validator as a parameter, the utility function can accept the `ScriptHash` directly and avoid scanning the transaction inputs.

Aiken

```
// Current
pub fn require_self_withdraw_with_rdmr(
  out_ref: OutputReference,
  Transaction { inputs, redeemers, .. }: Transaction,
  rdmr: Data,
) {
  list.any(
    inputs,
    fn(
      Input {
        output_reference,
        output: Output { address: Address { payment_credential, .. }, .. },
      },
    ) {
      if output_reference == out_ref {
        expect Script(skh) = payment_credential
        rdmr == must_get_rdmr_by_purpose(redeemers, Withdraw(Script(skh)))
      } else {
        False
      }
    },
  )
}
```

Recommendation

Approach 1 – Separate withdraw logic & inject ScriptHash (no input scan)

1. Separate the withdraw logic: create a new validator (ex. babel_fee_logic), and transfer the withdraw logic into it.

2. Inject its ScriptHash as a parameter: add the withdraw-logic ScriptHash as a parameter to the babel_fee validator.
3. Modify require_self_withdraw_with_rdmr: modify `require_self_withdraw_with_rdmr` to take the ScriptHash as argument instead of traversing all the inputs, and rename to match it's new function (eg. *require_withdraw_with_rdmr*).
4. Relocate NFT parameters: move `protocol_script_nft` and `protocol_config_nft` out of babel_fee validator and into the new babel_fee_logic validator.
5. Source hashes from `ProtocolScriptDatum`: read `babel_fee_skh` from the protocol_script_ref input instead of deriving it from cred (withdraw Credential).
6. Extend `ProtocolScriptDatum`: add `withdraw_logic_skh: ScriptHash` to `ProtocolScriptDatum` so all required SKHs are sourced from the protocol_script_ref input.

Approach 2 – Input-indexed redeemer (O(1) fetch)

1. use input index: Pass `babel_fee_in_idx` and fetch that input directly (no scan), then assert it's the targeted UTxO and matches the expected script hash.
2. Create Wrapper redeemer:
   - `pub type BabelFeeWithIdx { self_in_idx: Int, inner: BabelFeeRedeemer }`
3. Then compare inner as needed and use `self_in_idx` for O(1) fetch.

Impact

- Lower execution cost: Avoids an O(n) scan over tx.inputs; the check becomes O(1).
- Cheaper fees: Fewer execution units (CPU/mem) consumed → lower on-chain transaction fees when spending with many inputs.
- Simpler failure paths: Removes per-element pattern matching/equality checks, reducing unnecessary script work.

This removes unnecessary input traversal, optimizes execution (O(1) instead of O(n)), and modularizes the logic for greater clarity and maintainability.

Resolution

Alternative Optimization — Used input index list `input_idxs: List<Int>` to do O(k) lookup instead of scanning inputs; removes O(n) scan and input-layout coupling. (k < n)
Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[I 102] Split withdraw variants into separate validators

Level	Severity	Status
1	Informational	Acknowledged

Description

> `lib/babel_fee.ak` (withdraw path)

The current withdraw path handles multiple options/branches inside a single validator. This couples unrelated behaviors, increases blast radius for changes, and makes targeted testing/auditing harder.

By separating each withdraw option into its own validator (one validator per variant), we reduce coupling and make behavior boundaries explicit. Future changes to one path won't risk unintended effects on others, and test surfaces become much smaller.

Recommendation

- Extract: seperate each withdraw variant of the `BabelFeeRedeemer` into an individual validator.
- Route: in the babel_fee validator, call the `require_withdraw_with_rdmr` function with the corresponding argument.
- Centralize parameters: (Optional - for additional flexibility)
  - Extend ProtocolScriptDatum: add per-variant validator ScriptHash fields so routing uses the protocol_script_ref reference input as the single source of truth.
  - Route/config from reference input: read all SKHs/config via `protocol_script_ref` (ProtocolScriptDatum) instead of constructor args.

Impact

- Smaller test surface: Unit/integration tests target a single path; faster feedback and clearer failures.
- Lower change risk: Edits in one withdraw path don't affect others (reduced blast radius).
- Auditability: Reviewers can reason about each path in isolation; fewer interdependencies.
- Upgrade safety: Future logic tweaks require re-testing only the affected validator.

Related to: Informational 05 (self-withdraw validation refactor).

[I 103] Precompute Formatted Address

Level	Severity	Status
1	Informational	Resolved

Description

`TransferOutFromWanchain` withdrawal action creates the expected transaction metadata on-chain. `wanchain_refund_bech32_address: ByteArray` value used in this metadata is provided by `ProtocolConfigDatum`, this value further undergoes formatting on-chain using `format_address` function. This repetitive on-chain computation for transfer out transactions can be avoided, making the validator more efficient.

```
// lib/bridge_asset/transfer_wanchain.ak                                    Aiken

{
  ...

  let map_data =
      ""
        |> bytearray.concat(format_single_text("type", 4))
        |> bytearray.concat(
            builtin.serialise_data(constants.wanchain_withdraw_action_type),
          )
        |> bytearray.concat(format_single_text("smgID", 5))
        |> bytearray.concat(builtin.serialise_data(wanchain_smg_id))
        |> bytearray.concat(format_single_text("toAccount", 9))
        |> bytearray.concat(
            format_single_text(btc_receiver_address, to_account_length),
          )
        |> bytearray.concat(format_single_text("fromAccount", 11))
        |> bytearray.concat(format_address(wanchain_refund_bech32_address)) // <--
      provide formatted address through `ProtocolConfigDatum`
        |> bytearray.concat(format_single_text("tokenPairID", 11))
        |> bytearray.concat(
            builtin.serialise_data(constants.wanchain_token_pair_id),
          )

  ...
}

fn format_address(address: ByteArray) -> ByteArray {
  let length = bytearray.length(address)
  if length > constants.wanchain_address_length_limit {
    builtin.cons_bytearray(
```

```
32        constants.array_notation + 2,
33        format_single_text(
34          bytearray.slice(address, 0, constants.wanchain_address_length_limit - 1),
35          constants.wanchain_address_length_limit,
36        )
37          |> bytearray.concat(
38              format_single_text(
39                bytearray.slice(
40                  address,
41                  constants.wanchain_address_length_limit,
42                  length - 1,
43                ),
44                length - constants.wanchain_address_length_limit,
45              ),
46            ),
47      )
48    } else {
49      builtin.cons_bytearray(
50        constants.array_notation + 1,
51        format_single_text(address, length),
52      )
53    }
54  }
```

Recommendation

It is recommended to provide the formatted address value (`format_address(wanchain_refund_bech32_address)`) in `ProtocolConfigDatum`'s `wanchain_refund_bech32_address: ByteArray` field.

Resolution

Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[I 104] Redundant Redeemer Verification

Level	Severity	Status
1	Informational	Resolved

Description

```
// lib/babel_fees/utils.ak
let (
    babel_fee_total_in_amount,
    is_valid_spending_bridge_asset_rdmr,
    babel_fee_in_addr,
  ) =
    list.foldr(
      inputs,
      (0, False, None),
      fn(
        Input {
          output: Output {
            value,
            address: Address { payment_credential, .. } as address,
            ..
          },
          output_reference,
        },
        (acc, is_valid_spending_bridge_asset_rdmr, babel_fee_in_addr),
      ) {
        if payment_credential == Script(babel_fee_skh) {
          ...
        } else if payment_credential == Script(bridge_asset_skh) {
          let bridge_asset_spending_rdmr =
            utils.must_get_rdmr_by_purpose(redeemers, Spend(output_reference))
          (
            acc,
            bridge_asset_rdmr_verifier(bridge_asset_spending_rdmr), // <-- checks every bridge asset redeemer even when a match is found
            babel_fee_in_addr,
          )
        } else {
          (acc, is_valid_spending_bridge_asset_rdmr, babel_fee_in_addr)
        }
      },
    )

```

```
// lib/babel_fees/supply_float.ak
fn is_supplying_bridge_asset_rdmr(rdmr: Redeemer) -> Bool {
  expect SupplyToFloat { .. } = rdmr
  True
}
```

The `generic_babel_fee_validator` utility function ignores the past state of `is_valid_spending_bridge_asset_rdmr` and rechecks qualifying redeemers which can be expensive. Additional redeemer validations can be skipped once an expected redeemer instance is found, making the validation more efficient. The `bridge_asset` validator implicitly ensures that all of its redeemers in a transaction are of the same constructor (i.e. a transaction cannot have UTxOs being spent using different redeemer actions), thus `babel_fee` validator need not enforce it too by means of `bridge_asset_rdmr_verifier`.

Recommendation

To skip checking `bridge_asset_skh` output reference's redeemer if `is_valid_spending_bridge_asset_rdmr` is `True`.

```
if payment_credential == Script(babel_fee_skh) {                    Aiken
          ...
} else if !is_valid_spending_bridge_asset_rdmr && payment_credential ==
Script(bridge_asset_skh) {
  ...
}
```

Resolution

Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[I 105] Possibly Inaccurate Transfer Amount

Level	Severity	Status
1	Informational	Acknowledged

Description

```
// lib/bridge_asset/withdraw_float.ak                                    Aiken

let expected_transfer_amount =
  if total_supply != 0 || circulating_dtoken != 0 {
    total_amount * total_supply / circulating_dtoken
  } else {
    expect Some(Input { output: Output { datum, .. }, .. }) =
      list.find(
        inputs,
        fn(Input { output: Output { value, .. }, .. }) {
          utils.contains_nft(value, float_pool_nft)
        },
      )
    expect InlineDatum(dt) = datum
    expect FloatPoolDatum { total_supply, .. } = dt // ?! why fetch total_supply once
    again? (this is input total_supply, earlier output_total supply alone was
    available)
    total_supply // <-- incorrect exchange rate if there are additional withdrawals
  }
```

When both `total_supply` and `circulating_dtoken` are zero in the output `FloatPoolDatum`, the code assumes this withdrawal alone emptied the pool and then pulls `total_supply` from the input datum to set `expected_transfer_amount`. But if multiple withdrawals are occurring concurrently (some not via `bridge_asset`), `output.total_supply == 0` can be true even though other withdrawals burned additional dWBTC. In that case, using the input `total_supply` as the transfer amount misprices the conversion and can cause the transaction to fail.

Recommendation

Make the calculation aware of all simultaneous burns by basing the transfer amount on the proportion of this burn against the total burn in the block/tx context, e.g.: `expected_transfer_amount = total_supply * total_amount / total_dwbtc_burn_amount`.

Also pair this with the safe-guard from earlier findings (use `&&` for non-zero checks and avoid mixed units) to prevent divide-by-zero and unit errors.

[I 106] Redundant Boolean Return

Level	Severity	Status
1	Informational	Resolved

Description

```aiken
// lib/bridge_asset/supply_float.ak
pub fn spend(
  bridge_assets: List<(Int, Int)>,
  out_ref: OutputReference,
  Transaction { inputs, redeemers, .. }: Transaction,
  rdmr: Data,
) {
  list.any(
    bridge_assets,
    fn((in_idx, _)) {
      let Input {
        output: Output { address: Address { payment_credential, .. }, .. },
        output_reference,
      } = flist.get(inputs, in_idx)
      if output_reference == out_ref {
        expect Script(bridge_asset_skh) = payment_credential
        expect
          rdmr == utils.must_get_rdmr_by_purpose(
            redeemers,
            Withdraw(Script(bridge_asset_skh)),
          )
        True // <-- redundant boolean return
      } else {
        False
      }
    },
  )
}
```

Recommendation

Eliminate the redundant boolean return; rely on the `expect` checks and return the comparison directly:

```aiken
if output_reference == out_ref {
  expect Script(bridge_asset_skh) = payment_credential
  rdmr == utils.must_get_rdmr_by_purpose(
```

```
4       redeemers,
5       Withdraw(Script(bridge_asset_skh)),
6     )
7   } else {
8     False
9   }
```

Resolution

Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[I 107] Redundant wBTC check

Level	Severity	Status
1	Informational	Resolved

Description

```
// lib/bridge_asset/supply_float.ak                                    Aiken
  ...

  let output_wbtc =
    assets.quantity_of(out_value, env.wbtc_pid, env.wbtc_asset_name) // <-- redundant check
  let output_dwbtc =
    assets.quantity_of(out_value, dtoken_pid, dtoken_name)

  expect and {
      (in_addr == out_addr)?,
      (dtoken_rate_denom > 0)?,
      (output_dwbtc >= input_wbtc * dtoken_rate_denom / dtoken_rate_num + input_dwbtc)?,
      (out_value
        |> assets.without_lovelace
        |> assets.add(dtoken_pid, dtoken_name, -output_dwbtc)
        |> assets.add(env.wbtc_pid, env.wbtc_asset_name, -output_wbtc) // <-- redundant check
        |> assets.is_zero)?,
      (out_datum == InlineDatum(expected_output_datum))?,
      (out_reference_script == None)?,
    }
  output_idx_bitset

  ...
```

`SupplyToFloat` converts all WBTC in the input (no partial conversion), so the returned `bridge_asset` output can only contain dWBTC. Any residual WBTC from integer-division rounding is claimable by the router and therefore won't appear in the output. Consequently, asserting the absence of WBTC in the output is redundant.

Recommendation

Remove the redundant WBTC measurement and subtraction in the output value check:

- Drop `output_wbtc` and the `assets.add(env.wbtc_pid, env.wbtc_asset_name, -output_wbtc)` step.
- Keep the dWBTC conservation check and the other invariants.

Resolution
Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[I 108] Optimize On-chain Transaction CBOR Creation

Level	Severity	Status
1	Informational	Acknowledged

Description

To verify the transaction metadata hash, the `metadata_validation` validator reconstructs the transaction CBOR on-chain by serializing fields drawn from the script context `Transaction`. This is costly. Since the `TransferOutFromWanchain` transaction inputs are known ahead of time (before creating the reference UTxO at `script_data_hash_owner`) and remain stable if chosen deterministically, we can pass the pre-serialized CBOR of the inputs list via the redeemer to avoid recomputing it on-chain.

Concretely: provide the CBOR bytes for `inputs: List<Input>` in `MetadataValidationRedeemer`, and have the validator use these bytes directly when assembling the transaction CBOR for hash verification.

Recommendation

Provide the CBOR of the inputs list through `MetadataValidationRedeemer` and consume it in the validator to eliminate on-chain serialization work.

[I 109] Simplify Reference Input List Iteration

Level	Severity	Status
1	Informational	Resolved

Description

```
fn extract_script_data_hash(                                                    Aiken
  reference_inputs: List<Input>,
  script_data_hash_owner: Address,
) -> ByteArray {
  when reference_inputs is {
    [] -> fail
    [Input { output: Output { address, datum, .. }, .. }, ..xs] ->
      if address == script_data_hash_owner {
        when datum is {
          InlineDatum(dt) ->
            if dt is ScriptDataHashDatum {
              let ScriptDataHashDatum { script_data_hash } = dt
              script_data_hash
            } else {
              extract_script_data_hash(xs, script_data_hash_owner)
            }
          _ -> extract_script_data_hash(xs, script_data_hash_owner)
        }
      } else {
        extract_script_data_hash(xs, script_data_hash_owner)
      }
  }
}
```

Only one reference input from `script_data_hash_owner` is expected in the `TransferOutFromWanchain` transaction. We can simplify the function by asserting the datum shape when the first eligible input is found, instead of recursively scanning and branching on multiple cases.

Recommendation

```
fn extract_script_data_hash(                                                    Aiken
  reference_inputs: List<Input>,
  script_data_hash_owner: Address,
) -> ByteArray {
  when reference_inputs is {
    [] -> fail
    [Input { output: Output { address, datum, .. }, .. }, ..xs] ->
```

```
8         if address == script_data_hash_owner {
9           expect InlineDatum(dt) = datum
10          expect ScriptDataHashDatum { script_data_hash } = dt
11          script_data_hash
12        } else {
13          extract_script_data_hash(xs, script_data_hash_owner)
14        }
15    }
16  }
```

Resolution

Resolved in commit hash `ccba405d96b8406293c121d28a584c35ff7c21d6`

[I 110] Avoid per-UTxO scan in `bridge_asset.spend`

Level	Severity	Status
1	Informational	Deprioritized

Description

> lib/validators/bridge_asset.ak > lib/bridge_asset/supply_float.ak

`bridge_asset.spend` iterates over `bridge_assets` for every bridge_asset input UTxO to confirm we're calling this script's *withdraw* with the right redeemer/index:

```
list.any(                                                                    Aiken
  bridge_assets,
  fn((in_idx, _)) {
    let Input {
      output: Output { address: Address { payment_credential, .. }, .. },
      output_reference,
    } = flist.get(inputs, in_idx)
    if output_reference == out_ref {
      expect Script(bridge_asset_skh) = payment_credential
      expect
        rdmr == utils.must_get_rdmr_by_purpose(
          redeemers,
          Withdraw(Script(bridge_asset_skh)),
        )
      True
    } else {
      False
    }
  },
)
```

This couples `spend` to input layout and adds an unnecessary *O(n_bridge_assets)* pass even though `withdraw` already folds the same list and performs the substantive checks.

Recommendation

Approach 1 – Separate withdraw logic & inject `ScriptHash` (make `spend` O(1))

- Extract withdraw rules into a dedicated validator (e.g., `bridge_asset_logic`).
- Pass its `ScriptHash` into `bridge_asset` and replace the scan with one guard: `utils.require_withdraw_with_rdmr(out_ref, tx, Withdraw(Script(bridge_asset_logic_skh)))`.
- Move `protocol_script_nft` / `protocol_config_nft` parameters to the withdraw-logic validator.
- Move checks from `supply_float.spend` into the withdraw-logic validator.
- Use sorted, unique indices for `bridge_assets` (strictly increasing `in_idx`, unique `out_idx` via bitset) for deterministic *O(k)* withdraw iteration.

Approach 2 – Input-indexed redeemer (no scan, single validator)

- Add `self_in_idx: Int` beside `BridgeAssetRedeemer`.
- In `supply_float.spend`, fetch `tx.inputs[self_in_idx]` and assert: `output_reference == out_ref`, `payment_credential == Script(bridge_asset_skh)`, and redeemer equals `Withdraw(Script(bridge_asset_skh))`.
- Apply the same pattern to `WithdrawFromFloat` and `TransferOutFromWanchain` to avoid scans.
- Keep `output_reference == out_ref` to prevent lying indexes.

Impact

- Lower execution cost: remove per-UTxO scan; make `spend` constant-time.
- Cleaner modularity: business rules live in `withdraw`; `spend` just asserts correct binding.

Resolution

Not implemented. Team kept current scan in `bridge_asset.spend`; Revisit if execution units show `O(n)` scan becoming a bottleneck.

End of Report